

# GHOST CHAT ULTRA X

## Secure Messaging Platform

---

### Product Requirements Document (PRD) Software Requirements Specification (SRS)

---

*Version 1.0.0 — Industry Release Candidate*

**Classification: CONFIDENTIAL**

Date: June 2026

# Document Control

| Field          | Details                                          |
|----------------|--------------------------------------------------|
| Document Title | Ghost Chat Ultra X — PRD & SRS                   |
| Version        | 1.0.0 — Industry Release Candidate               |
| Classification | CONFIDENTIAL — Internal Use Only                 |
| Owner          | Product Engineering & Security Architecture Team |
| Review Cycle   | Quarterly or upon major threat-model update      |
| Approved By    | CTO, CISO, Lead Security Architect               |
| Last Updated   | June 2026                                        |
| Status         | Active — Phase 1 Development                     |

## Table of Contents

1. Executive Summary
2. Product Vision & Goals
3. Scope & Exclusions
4. Stakeholders & Users
5. System Architecture Overview
6. Network Layer Architecture
7. Cryptographic Architecture
8. Threat Model & Mitigations
9. Firewall Strategy (Multi-Layer)
10. Authentication & Authorization
11. Endpoint Hardening & Process Isolation
12. Secure Memory Management
13. Anti-Surveillance & Panic System
14. Functional Requirements (FR)
15. Non-Functional Requirements (NFR)
16. Data Flow Diagrams
17. API & Interface Specifications
18. Platform Support Matrix
19. Testing & Validation Strategy
20. Deployment Architecture
21. Compliance & Regulatory Alignment
22. Upgrade Roadmap (Tor / I2P Integration)
23. Glossary
24. Appendices

# 1. Executive Summary

Ghost Chat Ultra X (GCUX) is an industry-grade, zero-trust, end-to-end encrypted secure messaging platform designed for high-assurance environments where message confidentiality, sender anonymity, and anti-surveillance are mission-critical. GCUX is architected without any reliance on centralized servers, cloud infrastructure, or third-party trust anchors.

Version 1.0 operates exclusively over the host system's active network connection (Wi-Fi, Ethernet, mobile data) with a comprehensive multi-layer firewall strategy and a full STRIDE/MITRE ATT&CK-aligned threat model. The architecture is designed to be forward-compatible with anonymous overlay networks (Tor, I2P) as a Phase 2 upgrade without requiring a platform redesign.

**NOTE:** GCUX Version 1.0 does NOT route traffic through Tor or I2P. It uses the system's standard network stack hardened by multiple independent firewall layers, strict traffic filtering, and cryptographic protections at every hop. Tor/I2P integration is scoped to v2.0.

| Metric                   | Target                                                                 |
|--------------------------|------------------------------------------------------------------------|
| End-to-End Encryption    | Hybrid X25519 + ML-KEM-1024 (Post-Quantum)                             |
| Message Encryption       | ChaCha20-Poly1305 per-message unique keys                              |
| Identity Signatures      | ML-DSA-87 (Post-Quantum Digital Signatures)                            |
| Key Derivation           | HKDF-SHA3-512                                                          |
| Rekey Interval           | Every 50 messages or 5 minutes (whichever first)                       |
| Decryption Authorization | 6-factor: Key + Session + Device + Face + Liveness + Environment       |
| Persistent Storage       | Zero (RAM-only operation)                                              |
| Server Trust Required    | None                                                                   |
| Cloud Dependency         | None                                                                   |
| Firewall Layers          | 5 independent layers (Application, Host, Network, Transport, Protocol) |

## 2. Product Vision & Goals

### 2.1 Vision Statement

To provide two communicating parties absolute confidence that their messages cannot be read by any third party — regardless of network interception, server compromise, device seizure, or state-level adversary — through layered cryptographic, physical presence, and behavioral verification controls.

### 2.2 Strategic Goals

- G-01: Achieve post-quantum cryptographic resistance against current and foreseeable quantum computing threats.
- G-02: Eliminate server-side trust: no message plaintext ever leaves the endpoint.
- G-03: Gate decryption on physical presence verification to prevent relay attacks and unauthorized access.
- G-04: Deploy comprehensive multi-layer firewall controls to block network-level attacks.
- G-05: Implement full STRIDE/MITRE ATT&CK threat model coverage with documented mitigations.
- G-06: Support seamless forward upgrade to Tor/I2P overlay networks without protocol redesign.
- G-07: Maintain zero persistent data footprint — all sensitive data exists exclusively in protected RAM.

### 2.3 Success Metrics

| KPI                       | Target  | Measurement               |
|---------------------------|---------|---------------------------|
| Key Exchange Latency      | < 200ms | P95 on loopback & LAN     |
| Message Delivery Latency  | < 500ms | P95 end-to-end encrypted  |
| Face Verification Time    | < 800ms | Local inference, P95      |
| Memory Zeroization Time   | < 50ms  | Post-session cleanup      |
| Panic Wipe Completion     | < 200ms | From trigger to clean RAM |
| False Positive Face Lock  | < 0.1%  | 10,000-session test       |
| Network Attack Block Rate | > 99.9% | Pen-test suite            |
| Crash Recovery w/o Leak   | 100%    | Fault injection testing   |

## 3. Scope & Exclusions

### 3.1 In Scope — Version 1.0

- Peer-to-peer encrypted messaging over system network (any connected interface)
- Five-layer independent firewall architecture (Application, Host-based, Network, Transport, Protocol)
- Full STRIDE threat model with MITRE ATT&CK mapping and documented mitigations
- Hybrid classical + post-quantum key exchange and encryption
- Multi-factor decryption authorization (cryptographic + biometric + environmental)
- Local face recognition and liveness detection (on-device only, no cloud)
- Environment surveillance detection (secondary person, recording devices)
- Panic wipe system with multiple trigger vectors
- Zero persistent storage architecture
- Process isolation via Linux namespaces, AppArmor, and seccomp
- Secure memory management with multi-pass zeroization
- Screenshot/screen-capture resistance

### 3.2 Explicitly Out of Scope — Version 1.0

- Tor or I2P network routing (designed for v2.0 upgrade)
- Group messaging (v2.0 feature)
- File/media transfer (v1.5 feature)
- Voice or video calling
- Centralized user accounts or directories
- Server-side message storage or relay infrastructure
- Mobile platforms (v1.2 target)
- Windows platform (Linux-first; Windows in v1.3)

## 4. Stakeholders & User Personas

| Stakeholder           | Role                     | Concern                                         |
|-----------------------|--------------------------|-------------------------------------------------|
| Security Architect    | Design authority         | Cryptographic correctness, threat coverage      |
| Product Owner         | Requirements authority   | Feature completeness, roadmap alignment         |
| Lead Developer        | Implementation authority | Buildability, library availability, performance |
| CISO                  | Security sign-off        | Risk posture, compliance, audit trail           |
| QA/Pen-Test Team      | Validation authority     | Attack surface, regression, fuzzing             |
| End User (Journalist) | Primary operator persona | Ease of use under adversarial conditions        |
| End User (Researcher) | Primary operator persona | Data confidentiality, session isolation         |

## 5. System Architecture Overview

### 5.1 Architecture Principles

- Zero Trust: Every component verifies every other. No implicit trust.
- Defense in Depth: At least 3 independent controls guard every sensitive operation.
- Fail Secure: Any failure defaults to locked state, not open state.
- Minimal Footprint: Only required permissions and resources are acquired.
- Separation of Privilege: No single component can read plaintext alone.

### 5.2 High-Level Component Map

| Layer | Component                       | Responsibility                                           |
|-------|---------------------------------|----------------------------------------------------------|
| L0    | Firewall Stack (5 layers)       | Block all unauthorized inbound/outbound traffic          |
| L1    | Network Transport               | TCP/UDP socket management over system network            |
| L2    | Hybrid Key Exchange Engine      | X25519 + ML-KEM-1024 handshake, master secret derivation |
| L3    | Session Key Hierarchy           | Master → Session → Conversation → Message key derivation |
| L4    | Message Encryption Engine       | ChaCha20-Poly1305 per-message unique key+nonce           |
| L5    | Rekeying Scheduler              | Automatic key rotation every 50 messages or 5 minutes    |
| L6    | Identity Verification           | ML-DSA-87 digital signature verification                 |
| L7    | Biometric Gate                  | Face recognition + liveness detection (local only)       |
| L8    | Environment Detector            | Secondary person / recording device detection            |
| L9    | Decryption Authorization Engine | Orchestrates all verification checks before decryption   |
| L10   | Secure Memory Manager           | mlockall, protected buffers, multi-pass zeroization      |
| L11   | Panic/Wipe Controller           | Immediate key destruction and session termination        |
| L12   | Process Isolation Manager       | Linux namespaces, AppArmor, seccomp policy enforcement   |
| L13   | Display Renderer                | Wayland-based, blur/hide controls, screenshot resistance |



## 6. Network Layer Architecture

### 6.1 Network Transport Design

GCUX v1.0 operates over the host system's active network interface (Ethernet, Wi-Fi, or cellular) without any routing overlay. The application-level protocol stack is designed to be transport-agnostic, enabling a clean upgrade path to Tor Hidden Services or I2P garlic routing in v2.0.

### 6.2 Connection Bootstrap Protocol

1. Peer A generates an ephemeral connection token (256-bit random) and transmits it to Peer B over a pre-established secure side-channel.
2. Both peers open a direct TCP connection on a configurable port (default: 51820) with TLS 1.3 minimum.
3. Immediately upon TCP establishment, the GCUX hybrid handshake begins — no application data transits before handshake completion.
4. Post-handshake, the TLS layer becomes redundant (inner E2E encryption is stronger) but is maintained to resist traffic fingerprinting at the TLS record layer.
5. Session is established with a unique session ID bound to both device fingerprints.

### 6.3 Traffic Architecture

| Property        | Design Decision                                      | Rationale                                |
|-----------------|------------------------------------------------------|------------------------------------------|
| Packet Size     | Fixed 1280-byte padding to MTU                       | Prevents payload-length traffic analysis |
| Timing          | Randomized $\pm 0\text{--}250\text{ms}$ jitter added | Defeats timing correlation attacks       |
| Cover Traffic   | Dummy encrypted packets on idle                      | Prevents session-open/close inference    |
| Protocol Port   | Configurable (default 51820/TCP)                     | Avoid well-known port fingerprinting     |
| IP Header       | Minimal; no application fingerprints                 | Reduce OS-level metadata exposure        |
| DNS             | GCUX makes no DNS queries                            | Eliminate DNS-based traffic analysis     |
| Connection Mode | Direct peer-to-peer; no relay server                 | No server logs, no MITM surface          |

## 7. Cryptographic Architecture

### 7.1 Algorithm Selection Rationale

| Algorithm         | Role                           | Standard        | Quantum-Safe                |
|-------------------|--------------------------------|-----------------|-----------------------------|
| X25519            | Classical key exchange         | RFC 7748        | No (provides classical PFS) |
| ML-KEM-1024       | Post-quantum key encapsulation | NIST FIPS 203   | Yes                         |
| HKDF-SHA3-512     | Key derivation                 | RFC 5869        | Yes (SHA-3 family)          |
| ChaCha20-Poly1305 | Symmetric AEAD encryption      | RFC 8439        | Yes (256-bit symmetric)     |
| ML-DSA-87         | Digital signatures (identity)  | NIST FIPS 204   | Yes                         |
| BLAKE3            | Fast integrity hashing         | BLAKE3 spec     | Yes                         |
| CSPRNG            | Nonce/key randomness           | OS /dev/urandom | Yes                         |

### 7.2 Key Hierarchy

The key derivation hierarchy ensures that compromise at any single level does not expose keys at any other level:

| Level | Key Name                | Derived From                    | Lifetime                  |
|-------|-------------------------|---------------------------------|---------------------------|
| 0     | X25519 Ephemeral Secret | ECDH ephemeral keypairs         | Single handshake          |
| 0     | ML-KEM Shared Secret    | Post-quantum encapsulation      | Single handshake          |
| 1     | Master Secret           | HKDF-SHA3-512(X25519    ML-KEM) | Session                   |
| 2     | Session Key             | HKDF(Master Secret, session_id) | Session (5 min / 50 msg)  |
| 3     | Conversation Key        | HKDF(Session Key, conv_id)      | Conversation context      |
| 4     | Message Key             | HKDF(Session Key, msg_counter)  | Single message (use-once) |
| 4     | Nonce                   | CSPRNG fresh per message        | Single message (use-once) |

### 7.3 Forward Secrecy Guarantee

**REQUIREMENT MET:** Every session key is destroyed immediately after rotation. Old message keys are zeroized after display. An attacker who compromises the current session key gains access to at most 50 messages or 5 minutes of traffic — never historical messages from previous rekeying epochs.

### 7.4 Hybrid Quantum Resistance

The dual-path key exchange (classical X25519 + post-quantum ML-KEM-1024) means that even if the classical elliptic-curve algorithm is broken by a quantum adversary in the future, the

ML-KEM path independently provides 256-bit post-quantum security. Conversely, if ML-KEM contains an undiscovered flaw, the classical X25519 path maintains conventional security. Both paths must be broken simultaneously for the master secret to be compromised.

## 8. Threat Model & Mitigations

### 8.1 Threat Modeling Methodology

GCUX employs STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) threat categorization cross-referenced with MITRE ATT&CK for Enterprise and MITRE ATT&CK for Mobile. Each threat is assigned a severity rating using the DREAD model (Damage, Reproducibility, Exploitability, Affected Users, Discoverability).

### 8.2 STRIDE Threat Analysis & Mitigations

#### S — Spoofing Threats

| Threat ID | Attack                                 | MITRE ATT&CK | Severity | Mitigation                                                                                 |
|-----------|----------------------------------------|--------------|----------|--------------------------------------------------------------------------------------------|
| S-01      | Identity spoofing via forged handshake | T1534        | CRITICAL | ML-DSA-87 signatures on all messages; identity bound to long-term signing key              |
| S-02      | Session hijacking mid-stream           | T1557        | HIGH     | Session ID cryptographically bound to device fingerprint + handshake material              |
| S-03      | Replay attack with captured packets    | T1205        | HIGH     | Message counter + nonce uniqueness; replayed packets rejected with monotonic counter check |
| S-04      | Face spoofing via photo/video          | T1556        | HIGH     | Liveness detection: blink, eye movement, head motion; depth sensing where HW available     |

#### T — Tampering Threats

| Threat ID | Attack                                 | MITRE ATT&CK | Severity | Mitigation                                                                             |
|-----------|----------------------------------------|--------------|----------|----------------------------------------------------------------------------------------|
| T-01      | Packet modification in transit         | T1565.002    | CRITICAL | ChaCha20-Poly1305 AEAD; any bit-flip invalidates authentication tag, packet dropped    |
| T-02      | Key material tampering in memory       | T1055        | CRITICAL | mlockall() prevents swap; memory guard pages; integrity hash on key buffers            |
| T-03      | Binary tampering of GCUX executable    | T1195        | HIGH     | Code signing with verified hash on launch; AppArmor policy restricts self-modification |
| T-04      | Tampered system libraries (LD_PRELOAD) | T1574.006    | HIGH     | Static linking of critical crypto paths; LD_PRELOAD blocked by AppArmor policy         |

## R — Repudiation Threats

| Threat ID | Attack                                | MITRE ATT&CK | Severity | Mitigation                                                                          |
|-----------|---------------------------------------|--------------|----------|-------------------------------------------------------------------------------------|
| R-01      | Sender denies sending message         | T1562        | MEDIUM   | ML-DSA-87 per-message signatures; recipient retains cryptographic proof per-session |
| R-02      | Tampering with in-session message log | T1565        | MEDIUM   | BLAKE3 hash chain across session; any tampering breaks chain and triggers warning   |

## I — Information Disclosure Threats

| Threat ID | Attack                              | MITRE ATT&CK | Severity | Mitigation                                                                               |
|-----------|-------------------------------------|--------------|----------|------------------------------------------------------------------------------------------|
| I-01      | Network traffic interception (MITM) | T1557        | CRITICAL | Hybrid PQC encryption; even captured ciphertext is computationally infeasible to decrypt |
| I-02      | Memory dump of running process      | T1005        | CRITICAL | mlockall(); no-swap; guard pages; immediate zeroization after display                    |
| I-03      | Screenshot / screen recording       | T1113        | HIGH     | Wayland layer controls; blur on focus loss; recording software detection triggers hide   |
| I-04      | Cold boot attack on RAM             | T1006        | HIGH     | Multi-pass zeroization; encrypted memory regions; panic wipe on unexpected shutdown      |
| I-05      | Traffic metadata analysis           | T1040        | MEDIUM   | Fixed-size packets; random jitter; cover traffic; no DNS; no identifiable headers        |
| I-06      | Key material in core dump           | T1552        | HIGH     | RLIMIT_CORE = 0; prctl(PR_SET_DUMPABLE, 0); keys never in swappable pages                |
| I-07      | Swap file key exposure              | T1006        | HIGH     | mlockall(MCL_CURRENT   MCL_FUTURE) prevents any keying material from being swapped       |

## D — Denial of Service Threats

| Threat ID | Attack                                | MITRE ATT&CK | Severity | Mitigation                                                                      |
|-----------|---------------------------------------|--------------|----------|---------------------------------------------------------------------------------|
| D-01      | TCP SYN flood against GCUX port       | T1498.001    | HIGH     | SYN cookies; connection rate limiting at host firewall; iptables SYN rate rules |
| D-02      | Handshake exhaustion (ML-KEM amplif.) | T1499        | HIGH     | Max concurrent handshakes = 2; exponential backoff on failures; token challenge |

| Threat ID | Attack                                  | MITRE ATT&CK | Severity | Mitigation                                                                 |
|-----------|-----------------------------------------|--------------|----------|----------------------------------------------------------------------------|
| D-03      | Memory exhaustion via large messages    | T1499.002    | MEDIUM   | Max message size enforcement (4KB); message rate limiting per session      |
| D-04      | CPU exhaustion via crypto amplification | T1499        | MEDIUM   | Rate-limited handshakes; reject malformed packets before crypto processing |

## E — Elevation of Privilege Threats

| Threat ID | Attack                                  | MITRE ATT&CK | Severity | Mitigation                                                                  |
|-----------|-----------------------------------------|--------------|----------|-----------------------------------------------------------------------------|
| E-01      | GCUX process escape via buffer overflow | T1203        | CRITICAL | seccomp whitelist; AppArmor MAC; ASLR + PIE; stack canaries; fortify source |
| E-02      | Malicious plugin/extension injection    | T1559        | HIGH     | No plugin architecture in v1.0; all code paths statically analyzed          |
| E-03      | ptrace-based key extraction             | T1055.008    | CRITICAL | prctl(PR_SET_DUMPABLE,0); ptrace disabled via seccomp; yama ptrace_scope=3  |
| E-04      | Filesystem escape via path traversal    | T1083        | HIGH     | Linux mount namespace isolation; GCUX has no filesystem access at runtime   |

## 9. Firewall Strategy — Multi-Layer Architecture

GCUX implements five independent, composable firewall layers. Each layer provides a distinct control point. An attacker must defeat all five layers simultaneously to achieve unauthorized access.

### 9.1 Layer 1 — Application-Level Firewall (In-Process)

Implemented as an in-process packet validation and policy engine. Runs before any cryptographic processing.

| Rule                                      | Action                   | Rationale                                               |
|-------------------------------------------|--------------------------|---------------------------------------------------------|
| Source IP not in session allowlist        | DROP + LOG               | Only the pre-authorized peer IP is accepted             |
| Packet size < 64 bytes or > 1300 bytes    | DROP                     | Enforces fixed-size packet discipline                   |
| Message sequence counter out of order     | DROP + ALERT             | Detects replay or packet injection attempts             |
| Invalid ChaCha20-Poly1305 auth tag        | DROP + LOG               | AEAD authentication failure = tampering or wrong sender |
| Invalid ML-DSA-87 signature               | DROP + SESSION TERMINATE | Identity verification failure                           |
| Handshake without valid session token     | DROP                     | Prevents unauthorized connection bootstrap              |
| More than 2 concurrent handshake attempts | DROP + BACKOFF           | Mitigates handshake amplification DoS                   |

### 9.2 Layer 2 — Host-Based Firewall (iptables/nftables)

Kernel-level netfilter rules applied at the network stack level, independent of the GCUX process.

| Chain   | Rule                                           | Action                                   |
|---------|------------------------------------------------|------------------------------------------|
| INPUT   | State ESTABLISHED,RELATED                      | ACCEPT (existing connections)            |
| INPUT   | TCP dport 51820 (GCUX port) from peer IP       | ACCEPT (authorized peer only)            |
| INPUT   | ALL other inbound to port 51820                | DROP                                     |
| INPUT   | TCP SYN rate > 10/min from any single IP       | DROP (SYN flood protection)              |
| INPUT   | All other unsolicited inbound                  | DROP (default deny)                      |
| OUTPUT  | From GCUX process UID to peer IP:51820         | ACCEPT                                   |
| OUTPUT  | From GCUX process UID to any other destination | DROP (GCUX is network-isolated)          |
| FORWARD | All                                            | DROP (this is an endpoint, not a router) |

| Chain                                                                                                                                                                                                                        | Rule | Action |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|--------|
| <b>WARNING:</b> The iptables/nftables rules are applied on process launch and enforced with immutable attribute (chattr +i on rule snapshot). The GCUX process does not hold CAP_NET_ADMIN and cannot modify firewall rules. |      |        |

### 9.3 Layer 3 — Network-Level Firewall (Router/Gateway)

Recommendations for deployment-environment network infrastructure. GCUX publishes a required gateway ACL configuration:

| Control                   | Recommendation                                                                        |
|---------------------------|---------------------------------------------------------------------------------------|
| Inbound ACL               | Allow only established/related traffic and authorized peer IPs to reach GCUX host     |
| Outbound ACL              | GCUX host may only initiate outbound TCP to the authorized peer IP on port 51820      |
| ICMP                      | Allow ICMP type 3 (unreachable) and type 11 (TTL exceeded) only; block all other ICMP |
| DNS filtering             | GCUX makes no DNS queries; any DNS from the GCUX host is anomalous — alert            |
| HTTP/HTTPS from GCUX host | Block entirely — GCUX requires no internet access beyond peer connection              |
| Network segmentation      | GCUX host should be on a dedicated VLAN isolated from other hosts                     |
| Logging                   | Full NetFlow/sFlow logging to SIEM; retention per security policy                     |

### 9.4 Layer 4 — Transport Security Layer (TLS 1.3)

An outer TLS 1.3 wrapper is maintained on the TCP connection. While the inner GCUX E2E encryption provides stronger security guarantees, the TLS layer provides:

- Additional traffic fingerprinting resistance (GCUX traffic appears as TLS)
- Mutual TLS certificate pinning (both peers validate pinned certificate fingerprints)
- TLS 1.3 0-RTT is disabled (prevents replay attacks at TLS layer)
- Only TLS\_CHACHA20\_POLY1305\_SHA256 and TLS\_AES\_256\_GCM\_SHA384 cipher suites permitted
- Certificate transparency log checking on initial session establishment

### 9.5 Layer 5 — Process Isolation Firewall (seccomp/AppArmor)

The GCUX process operates under a strict seccomp system call whitelist and AppArmor Mandatory Access Control profile, forming a software-defined firewall around the process itself.

| Control               | Mechanism   | Effect                                                               |
|-----------------------|-------------|----------------------------------------------------------------------|
| System call whitelist | seccomp BPF | Only approved syscalls permitted; all others return EPERM or SIGKILL |



| Control                 | Mechanism          | Effect                                                                   |
|-------------------------|--------------------|--------------------------------------------------------------------------|
| File access restriction | AppArmor profile   | GCUX cannot read/write any filesystem path except its own binary         |
| Network restriction     | AppArmor + seccomp | GCUX can only use pre-established sockets; cannot create new raw sockets |
| Ptrace block            | seccomp + Yama     | No process can ptrace GCUX; PR_SET_DUMPABLE=0 prevents core dumps        |
| Privilege drop          | setuid/setgid drop | GCUX drops all capabilities immediately after socket bind                |
| Mount namespace         | Linux namespaces   | GCUX cannot access or enumerate host filesystem mounts                   |
| PID namespace           | Linux namespaces   | GCUX cannot enumerate or signal other processes                          |

## 10. Authentication & Authorization

### 10.1 Identity Model

GCUX uses a keypair-based identity model. There are no usernames, passwords, or central identity providers. Each user is uniquely identified by a long-term ML-DSA-87 signing keypair generated at first launch and stored in an encrypted key vault protected by local biometrics.

### 10.2 Decryption Authorization Matrix

Decryption of any message requires ALL six factors to pass simultaneously. Failure of any single factor prevents decryption and triggers appropriate response:

| Factor                 | Mechanism                                             | Failure Response                     |
|------------------------|-------------------------------------------------------|--------------------------------------|
| F-1: Cryptographic Key | Correct ML-KEM-1024 session key derived via handshake | Packet silently discarded            |
| F-2: Session State     | Valid session ID matching current session binding     | Connection terminated                |
| F-3: Device Binding    | Device fingerprint embedded in session material       | Session key invalidated              |
| F-4: Face Verification | Local face recognition with authorized face template  | Session locked until reverified      |
| F-5: Liveness Check    | Blink/eye movement/head motion detection              | Session locked; extended fail = wipe |
| F-6: Environment Check | No secondary person; no recording device detected     | Immediate wipe + disconnect          |

### 10.3 Session Lifecycle

6. User launches GCUX — face + liveness verification required to unlock key vault.
7. User initiates or accepts connection — hybrid handshake performed.
8. Session established — all 6 authorization factors active.
9. Message received — authorization engine re-verifies factors before decryption.
10. Rekeying event (50 messages or 5 min) — old keys destroyed, new keys derived.
11. Session end — all key material multi-pass zeroized; RAM buffers cleared.

## 11. Endpoint Hardening & Process Isolation

### 11.1 Compile-Time Hardening

| Flag / Feature                | Purpose                                                  |
|-------------------------------|----------------------------------------------------------|
| -fstack-protector-strong      | Stack buffer overflow detection via canary values        |
| -D_FORTIFY_SOURCE=2           | Runtime bounds checking on string/memory operations      |
| -fPIE + -pie                  | Position Independent Executable for ASLR effectiveness   |
| -Wl,-z,relro -Wl,-z,now       | Read-only relocations; full RELRO prevents GOT overwrite |
| CFI (Clang CFI)               | Control Flow Integrity prevents ROP/JOP chain attacks    |
| Static linking of crypto libs | Prevents malicious shared library substitution           |
| LTO (Link-Time Optimization)  | Enables cross-module CFI and dead code elimination       |

### 11.2 Runtime Hardening

| Mechanism           | Implementation                                                  | Protection                                                        |
|---------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|
| ASLR                | Kernel-enforced (require /proc/sys/kernel/randomize_va_space=2) | Randomizes memory layout; defeats hardcoded address exploits      |
| Namespace isolation | clone(CLONE_NEWNET   CLONE_NEWPID   CLONE_NEWNS)                | Network, PID, and mount isolation from host                       |
| seccomp BPF         | Whitelist of ~25 syscalls only                                  | All other syscalls blocked; prevents exploit pivoting             |
| AppArmor            | Custom enforcement profile                                      | MAC policy: no filesystem access, limited network, no raw sockets |
| Capability drop     | cap_drop ALL after init; cap_add only CAP_NET_BIND_SERVICE      | Minimal privilege operation after startup                         |
| Yama ptrace_scope   | Set to 3 (no ptrace at all)                                     | Prevents debugger/injector from attaching to process              |
| ulimit              | RLIMIT_CORE=0; RLIMIT_NOFILE minimal                            | No core dumps; minimal file descriptor surface                    |

## 12. Secure Memory Management

### 12.1 Memory Allocation for Sensitive Data

- All key material allocated via custom secure allocator backed by `mmap(MAP_ANONYMOUS | MAP_PRIVATE)`.
- `mlockall(MCL_CURRENT | MCL_FUTURE)` called at startup — prevents all process memory from being swapped to disk.
- Guard pages (`PROT_NONE`) placed before and after each key buffer — any overread/overwrite raises `SIGSEGV`.
- Sensitive buffers tagged in process memory map; integrity BLAKE3 hash maintained for detection of out-of-band writes.

### 12.2 Zeroization Protocol

All sensitive memory is destroyed using a 3-pass zeroization procedure before deallocation:

| Pass   | Pattern                                | Purpose                                                             |
|--------|----------------------------------------|---------------------------------------------------------------------|
| Pass 1 | 0x00000000 (all zeros)                 | Overwrite data with zeros                                           |
| Pass 2 | CSPRNG random bytes                    | Prevent freeze-and-recover attacks that detect zero patterns        |
| Pass 3 | 0x00000000 (all zeros)                 | Final clean state; compiler cannot optimize away (volatile pointer) |
| Final  | <code>munmap()</code> + memory release | Return memory to OS; cannot be re-read by GCUX                      |

**WARNING:** The `volatile` keyword alone is insufficient. GCUX uses `explicit_bzero()` or equivalent that is guaranteed not to be optimized out by the compiler. Memory barrier instructions are inserted to ensure write ordering.

### 12.3 Decrypted Plaintext Lifetime

Decrypted message plaintext follows this strict lifecycle:

12. Plaintext produced in a dedicated guard-paged buffer.
13. Passed directly to the display renderer via a shared memory region with restricted permissions.
14. User views the message.
15. On message hide, focus loss, or session lock: display renderer zeroes its copy; source buffer is immediately zeroized.
16. Total plaintext lifetime in RAM: typically < 30 seconds; hard maximum 60 seconds before forced zeroization.

## 13. Anti-Surveillance & Panic System

### 13.1 Environment Surveillance Detection

| Threat                    | Detection Method                                 | Response                                            |
|---------------------------|--------------------------------------------------|-----------------------------------------------------|
| Secondary person in view  | Face detection: > 1 face in frame                | Immediate: hide plaintext, lock session, alert user |
| Recording device (phone)  | Object detection classifier (local inference)    | Immediate: panic wipe + disconnect                  |
| Secondary camera detected | OS device enumeration delta monitoring           | Lock session pending user review                    |
| Screen recording software | Process name + window compositor API check       | Hide display content, alert user                    |
| Virtual machine / sandbox | CPUID checks, timer resolution, hypervisor flags | Refuse to launch; display warning                   |
| Debugger attached         | ptrace self-check; /proc/self/status TracerPid   | Immediate panic wipe and exit                       |
| Unauthorized USB device   | udev event monitoring for new HID/storage        | Alert; optional auto-lock                           |

### 13.2 Panic Wipe Triggers & Actions

| Trigger                        | Type               | Action Sequence                                        |
|--------------------------------|--------------------|--------------------------------------------------------|
| Hotkey (Ctrl+Alt+W)            | User-initiated     | Key wipe → buffer wipe → disconnect → exit             |
| Second face detected           | Automatic          | Key wipe → buffer wipe → disconnect → exit             |
| Recording device detected      | Automatic          | Key wipe → buffer wipe → disconnect → exit             |
| Liveness check failure (3x)    | Automatic          | Session lock → key wipe if not cleared in 30s          |
| Debugger attach detected       | Automatic          | Immediate key wipe → buffer wipe → SIGKILL self        |
| Unexpected process termination | Automatic (atexit) | atexit handlers: key wipe → buffer wipe                |
| SIGTERM / SIGINT received      | OS signal          | Graceful wipe sequence before exit                     |
| Hardware token removed (v1.5+) | Hardware event     | Immediate session lock; wipe if not re-inserted in 10s |

**CRITICAL:** Panic wipe is designed to complete within 200ms. All key material is zeroed before any session close message is sent to the peer. The peer will detect an unexpected connection termination and lock their own session.

## 14. Functional Requirements

### 14.1 Core Messaging

| ID     | Requirement                                                                                 | Priority |
|--------|---------------------------------------------------------------------------------------------|----------|
| FR-001 | GCUX SHALL establish an encrypted P2P session via hybrid X25519+ML-KEM-1024 handshake.      | MUST     |
| FR-002 | GCUX SHALL encrypt every message with a unique per-message ChaCha20-Poly1305 key and nonce. | MUST     |
| FR-003 | GCUX SHALL sign every outbound message with the sender's ML-DSA-87 private key.             | MUST     |
| FR-004 | GCUX SHALL verify the ML-DSA-87 signature of every inbound message before processing.       | MUST     |
| FR-005 | GCUX SHALL automatically rekey (rotate session key) every 50 messages or 5 minutes.         | MUST     |
| FR-006 | GCUX SHALL immediately destroy all previous key material upon rekeying.                     | MUST     |
| FR-007 | GCUX SHALL support a maximum message size of 4096 bytes (configurable up to 8192 bytes).    | MUST     |
| FR-008 | GCUX SHALL pad all network packets to 1280 bytes to prevent payload-length analysis.        | MUST     |
| FR-009 | GCUX SHALL add random timing jitter (0–250ms) to all outbound transmissions.                | MUST     |
| FR-010 | GCUX SHALL transmit cover traffic (dummy encrypted packets) during idle periods.            | SHOULD   |

### 14.2 Authentication & Biometrics

| ID     | Requirement                                                                                         | Priority |
|--------|-----------------------------------------------------------------------------------------------------|----------|
| FR-020 | GCUX SHALL require successful face verification before any session key vault access.                | MUST     |
| FR-021 | GCUX SHALL perform liveness detection (blink, eye movement, head motion) to prevent photo spoofing. | MUST     |
| FR-022 | GCUX SHALL perform all biometric processing locally with no data leaving the device.                | MUST     |
| FR-023 | GCUX SHALL lock the session upon face verification failure and require re-verification.             | MUST     |
| FR-024 | GCUX SHALL trigger panic wipe after 3 consecutive liveness verification failures.                   | MUST     |
| FR-025 | GCUX SHALL detect secondary faces and immediately lock/wipe.                                        | MUST     |

### 14.3 Memory & Data Handling

| ID     | Requirement                                                                | Priority |
|--------|----------------------------------------------------------------------------|----------|
| FR-030 | GCUX SHALL store NO data on disk — all session data is RAM-only.           | MUST     |
| FR-031 | GCUX SHALL call mlockall(MCL_CURRENT MCL_FUTURE) at startup.               | MUST     |
| FR-032 | GCUX SHALL perform 3-pass zeroization of all key material on session end.  | MUST     |
| FR-033 | GCUX SHALL zeroize plaintext buffers within 60 seconds of message display. | MUST     |
| FR-034 | GCUX SHALL set RLIMIT_CORE=0 to prevent core dump creation.                | MUST     |
| FR-035 | GCUX SHALL use guard pages around all key buffers.                         | MUST     |

## 15. Non-Functional Requirements

| ID      | Category        | Requirement                                       | Target                        |
|---------|-----------------|---------------------------------------------------|-------------------------------|
| NFR-001 | Performance     | Key exchange handshake completion time            | < 200ms P95                   |
| NFR-002 | Performance     | End-to-end message delivery (LAN)                 | < 500ms P95                   |
| NFR-003 | Performance     | Face verification time (local inference)          | < 800ms P95                   |
| NFR-004 | Performance     | Panic wipe completion time                        | < 200ms                       |
| NFR-005 | Performance     | Memory zeroization time (256KB key material)      | < 50ms                        |
| NFR-006 | Security        | No key material in pagefile / swap at any time    | 100% guaranteed               |
| NFR-007 | Security        | Core dump prevention                              | 100% guaranteed               |
| NFR-008 | Security        | ASLR + PIE on all compiled binaries               | 100% mandatory                |
| NFR-009 | Reliability     | Panic wipe success rate on trigger                | > 99.99%                      |
| NFR-010 | Reliability     | Message delivery success rate (healthy network)   | > 99.5%                       |
| NFR-011 | Usability       | Session establishment (user action to chat ready) | < 5 seconds                   |
| NFR-012 | Availability    | GCUX process uptime during session                | > 99.9%                       |
| NFR-013 | Maintainability | Crypto algorithm swap without protocol break      | Modular design; < 2 dev-weeks |
| NFR-014 | Scalability     | Max simultaneous peer sessions                    | 2 (v1.0); 10 (v2.0 roadmap)   |



## 16. Data Flow — Message Send & Receive

### 16.1 Message Send Flow

| Step | Component                | Action                                                                |
|------|--------------------------|-----------------------------------------------------------------------|
| 1    | UI Layer                 | User types message; plaintext held in guard-paged RAM buffer          |
| 2    | Authorization Engine     | Verify: session valid + face present + liveness + clean environment   |
| 3    | Message Key Derivation   | HKDF(session_key, msg_counter) → message_key; fresh CSPRNG nonce      |
| 4    | Signing Engine           | Compute ML-DSA-87 signature over plaintext    metadata                |
| 5    | Encryption Engine        | ChaCha20-Poly1305 encrypt(message_key, nonce, plaintext    signature) |
| 6    | Packet Assembler         | Pad to 1280 bytes; add timing jitter (0–250ms delay)                  |
| 7    | Layer-1 App Firewall     | Validate outbound packet against policy before socket write           |
| 8    | TLS 1.3 Layer            | Encrypt packet under TLS record layer                                 |
| 9    | Host Firewall (iptables) | Kernel validates outbound matches GCUX UID + authorized peer IP rule  |
| 10   | Network Transmission     | Packet transmitted over system network to peer                        |

### 16.2 Message Receive Flow

| Step | Component                | Action                                                           |
|------|--------------------------|------------------------------------------------------------------|
| 1    | Network Stack            | Packet received from network interface                           |
| 2    | Host Firewall (iptables) | Kernel validates inbound: source IP, port, state                 |
| 3    | TLS 1.3 Layer            | Decrypt and validate TLS record                                  |
| 4    | Layer-1 App Firewall     | Validate: packet size, sequence counter, source session binding  |
| 5    | Message Validation       | Verify ChaCha20-Poly1305 authentication tag; if fail: DROP       |
| 6    | Authorization Engine     | Re-verify: session valid + face present + liveness + environment |
| 7    | Decryption Engine        | Decrypt with message_key derived from current session + counter  |
| 8    | Signature Verification   | Verify ML-DSA-87 signature; if fail: DROP + alert                |
| 9    | Display Renderer         | Plaintext passed to Wayland-protected display buffer             |
| 10   | Plaintext Lifecycle      | Display → max 60s → forced zeroization of plaintext buffer       |

## 17. Platform Support Matrix

| Platform                 | Status           | Notes                                                      |
|--------------------------|------------------|------------------------------------------------------------|
| Linux (x86_64) — Wayland | PRIMARY — v1.0   | Full feature support including display security controls   |
| Linux (x86_64) — X11     | SECONDARY — v1.0 | Screenshot resistance limited; recommend Wayland migration |
| Linux (ARM64)            | PLANNED — v1.1   | Raspberry Pi / SBC deployment target                       |
| macOS                    | PLANNED — v1.2   | Requires platform-specific secure memory and display APIs  |
| Windows 11               | PLANNED — v1.3   | VirtualSecureMode; WDAC for process isolation equivalent   |
| Android                  | PLANNED — v2.0   | Kotlin/JNI bridge; TEE-backed key storage                  |
| iOS                      | PLANNED — v2.0   | Swift; Secure Enclave for key storage                      |

## 18. Testing & Validation Strategy

### 18.1 Test Categories

| Test Type               | Scope                                             | Tool/Method                     | Pass Criteria                      |
|-------------------------|---------------------------------------------------|---------------------------------|------------------------------------|
| Unit Tests              | Each crypto function, zeroization, key derivation | Rust test framework / pytest    | 100% pass; known-answer tests      |
| Integration Tests       | Full handshake, message send/receive, rekey       | Custom test harness             | All flows complete without leak    |
| Penetration Testing     | Full attack surface: network, process, memory     | External red team + internal    | No critical findings unmitigated   |
| Fuzzing                 | Packet parser, message handler, handshake engine  | libFuzzer / AFL++               | 72-hour run with no critical crash |
| Memory Leak Testing     | Session lifecycle, panic wipe, normal exit        | Valgrind, AddressSanitizer      | Zero sensitive data post-session   |
| Firewall Rule Testing   | All 5 firewall layers against attack suite        | nmap, hping3, custom scripts    | >99.9% block rate                  |
| Biometric Spoofing Test | Photo attacks, video replay, mask attacks         | Certified liveness test dataset | < 0.1% false acceptance rate       |
| Regression Suite        | All critical paths on every code change           | CI/CD pipeline                  | 100% pass required for merge       |

## 19. Compliance & Regulatory Alignment

| Standard / Framework | Relevance                           | GCUX Alignment                                                         |
|----------------------|-------------------------------------|------------------------------------------------------------------------|
| NIST SP 800-175B     | Cryptographic standards guidance    | All algorithms selected from NIST-approved or NIST PQC finalist list   |
| NIST FIPS 203        | ML-KEM (post-quantum KEM)           | ML-KEM-1024 implemented per specification                              |
| NIST FIPS 204        | ML-DSA (post-quantum signatures)    | ML-DSA-87 implemented per specification                                |
| NIST SP 800-38D      | GCM mode AEAD guidance              | ChaCha20-Poly1305 used (equivalent AEAD; stronger for software)        |
| OWASP MASVS          | Mobile/desktop application security | MASVS-CRYPTO, MASVS-NETWORK, MASVS-RESILIENCE controls applied         |
| MITRE ATT&CK         | Threat modeling framework           | All applicable TTPs mapped with documented mitigations (Section 8)     |
| STRIDE               | Threat categorization methodology   | Full STRIDE analysis completed (Section 8)                             |
| GDPR Article 25      | Privacy by design                   | Zero persistent data; no telemetry; no cloud; minimal data collection  |
| ISO/IEC 27001        | Information security management     | Key management, access control, and incident response controls aligned |

## 20. Upgrade Roadmap — Tor / I2P Integration (v2.0)

The v1.0 architecture is explicitly designed to support Tor Hidden Services (v3 onion) and I2P garlic routing as a transport layer upgrade without requiring any changes to the cryptographic or application layers above.

### 20.1 Why the Architecture is Tor-Ready

- The network transport layer is abstracted behind a SocketProvider interface; swapping TCP sockets for SOCKS5 (Tor) or I2P SAMv3 connections requires only a transport adapter implementation.
- GCUX already makes no DNS queries — compatible with Tor's DNS-over-circuit model.
- Fixed-size packet padding and cover traffic are already implemented — Tor benefits from these existing controls.
- No IP addresses are embedded in GCUX application messages — zero refactoring for hidden service operation.

### 20.2 v2.0 Planned Changes

| Change                     | Description                                                | Impact                              |
|----------------------------|------------------------------------------------------------|-------------------------------------|
| Transport Provider Swap    | Replace direct TCP with SOCKS5 → Tor circuit               | Low: adapter pattern only           |
| Onion Address Distribution | Peer identity = .onion v3 address derived from signing key | Medium: key derivation extension    |
| I2P SAMv3 Support          | Optional I2P transport as Tor fallback                     | Low: second adapter implementation  |
| Rendezvous Protocol        | Secure .onion address exchange via out-of-band channel     | Low: existing side-channel pattern  |
| Circuit Isolation          | Use separate Tor circuits per session                      | Low: Tor control port configuration |

**REQUIREMENT MET:** None of the cryptographic, memory management, biometric, or firewall layers require modification for Tor integration. The separation of concerns in the v1.0 design ensures that the Tor upgrade is purely an additive network transport change.

## 21. Glossary

| Term              | Definition                                                                                 |
|-------------------|--------------------------------------------------------------------------------------------|
| AEAD              | Authenticated Encryption with Associated Data — encryption that also verifies integrity    |
| ASLR              | Address Space Layout Randomization — OS randomizes memory addresses to defeat exploits     |
| ChaCha20-Poly1305 | A high-speed AEAD cipher; preferred over AES-GCM for software implementations              |
| Forward Secrecy   | Property where compromise of a current key does not expose past communications             |
| HKDF              | HMAC-based Key Derivation Function — derives multiple keys from one master secret          |
| ML-DSA-87         | Module-Lattice Digital Signature Algorithm — NIST FIPS 204 post-quantum signature standard |
| ML-KEM-1024       | Module-Lattice Key Encapsulation Mechanism — NIST FIPS 203 post-quantum KEM standard       |
| mlockall()        | Linux syscall that locks all current and future process memory pages to RAM (no swap)      |
| Panic Wipe        | Immediate cryptographic key and buffer destruction triggered by threat detection           |
| PQC               | Post-Quantum Cryptography — algorithms resistant to attacks by quantum computers           |
| seccomp BPF       | Linux kernel mechanism to restrict the system calls a process is permitted to make         |
| STRIDE            | Threat modeling methodology: Spoofing, Tampering, Repudiation, Info Disclosure, DoS, EoP   |
| X25519            | Elliptic-curve Diffie-Hellman key exchange using Curve25519 — RFC 7748                     |
| Zeroization       | Secure overwriting of memory to destroy sensitive data, preventing recovery                |

## Appendix A — Approved Algorithm Reference

| Category             | Algorithm         | Standard      | Key Size     | Security Level    |
|----------------------|-------------------|---------------|--------------|-------------------|
| KEM (classical)      | X25519            | RFC 7748      | 256-bit      | 128-bit classical |
| KEM (post-quantum)   | ML-KEM-1024       | NIST FIPS 203 | 1568-byte pk | 256-bit quantum   |
| Key Derivation       | HKDF-SHA3-512     | RFC 5869      | N/A          | 256-bit effective |
| Symmetric Encryption | ChaCha20-Poly1305 | RFC 8439      | 256-bit      | 256-bit symmetric |
| Digital Signatures   | ML-DSA-87         | NIST FIPS 204 | 4896-byte pk | 256-bit quantum   |
| Hashing              | BLAKE3            | BLAKE3 spec   | N/A          | 256-bit output    |
| Random Generation    | OS CSPRNG         | /dev/urandom  | N/A          | Full entropy      |
| TLS Outer Layer      | TLS 1.3           | RFC 8446      | N/A          | Forward secrecy   |

## Appendix B — Security Assumptions

The following assumptions must hold for GCUX security guarantees to be valid:

- 17. The host operating system kernel is not compromised.
- 18. The hardware is not backdoored (no hardware keylogger, no malicious firmware).
- 19. The biometric enrollment was performed in a secure, unobserved environment.
- 20. The long-term signing keypair was generated on a secure device.
- 21. The peer connection initiation (sharing the initial session token) was performed securely.
- 22. The user does not voluntarily share their keys or session credentials.

Security engineering operates on the principle of increasing attack cost and difficulty, not absolute impossibility. GCUX is designed to make all known attacks computationally infeasible or operationally impractical under the above assumptions.